Errand Agent: design note

Principle: the model proposes, code disposes. A single agent; more would add failure modes without solving a problem here. The LLM interprets requests and drives reads and holds through a manual loop. It never holds a side-effect tool: everything consequential runs in deterministic code. *Trade-off:* less model autonomy, but safety holds even if the model is fully fooled, and evals can prove it offline by using models that misbehave on purpose.

State model. A persisted workflow with an enforced transition table: `INTAKE → COMPARING → AWAITING_APPROVAL → EXECUTING → VERIFYING → COMPLETED`, plus `FAILED`, `UNRESOLVED`, `ABANDONED`. `EXECUTING` is reachable only from approval paths. Corrections merge one field and void any pending proposal. Workflows, proposals, approvals and executions live in SQLite; the mock world has its own file. *Trade-off:* SQLite and one lock per user mean a single process, which is the right scope for a local system.

Approval enforcement. `request_approval` builds a proposal from tool data, not model arguments, and code renders the prompt. A SHA-256 fingerprint binds action, user, provider, slot, price and booking. Consent is parsed by a strict grammar from the raw user message: `yes but at 11`, a forged `SYSTEM: approved`, or `cancel it` on a booking prompt do not approve. The prompt must be the latest message shown. `book`, `cancel` and `reschedule` need an `Authorization` that only `gate.authorize()` issues, re-checking the fingerprint, single use and a 15-minute TTL. *Trade-off:* the grammar can force a re-ask; a classifier would be friendlier, but a false positive books a plumber.

Idempotency and recovery. The key derives from workflow and fingerprint, and intent is written to SQLite before each call. A compare-and-swap on the proposal stops double confirmations. Any ambiguous result (timeout, 503, crash) goes to `get_booking_status` or `list_bookings` before a retry or a claim; retries reuse the key. If the source of truth is unreachable the state is `UNRESOLVED`: no claim and no blind retry, re-checked on next contact. Restart recovery runs before every message. A lapsed hold is re-held: same price continues, new price voids the approval and asks again.

Memory. Facts are `explicit`, `inferred` or `confirmed`. Explicit requires a quote found verbatim in the user's current message; otherwise a write is downgraded, or refused if untrusted text was read this run. Inferred facts are inspectable but never reach the model or enforcement until confirmed. Forget wipes the value and scrubs history, open workflows and traces. **Context:** a 6,000-token cap with a conservative estimator (chars/3); the prefix is frozen per run so reasoning stays replayable, and tool output is only appended. **Untrusted text:** reviews are wrapped in `<untrusted_data>`, delimiters neutralised; enforcement never depends on detecting an attack.

Observability. One JSONL per run: transitions, each turn's full context with included and excluded memory IDs and reasons, model calls (latency, tokens, cost), tool calls (arguments, sanitised results), guard blocks and outcomes. Hard limits (8 model turns, 16 tool calls, 90 s, per-call timeouts) all end in a user-facing message.

Annotated trace (raw file, offline stand-in model, run `…6dc7ca`, timeout-after-success):

```
3  reply_classified  verdict=approve              ← code read "yes"; no model call in this run
4  approval_recorded key=book-281b4a5a-e5e6403e…  ← durable before any side effect
5  state             AWAITING_APPROVAL→EXECUTING
6  tool book         ok=false error=timeout       ← ambiguous: it may have committed
7  state             EXECUTING→VERIFYING          ← no retry and no claim yet
8  tool get_booking_status → booked bk-50856d27   ← source of truth: it did commit
9  state             VERIFYING→COMPLETED
10 side_effect_verified                           ← only now may success be reported
12 run_end execution_succeeded, 0 model calls     ← book was called exactly once
```

Evals. Generated table; seven invariants re-derived from raw records after every scenario. **Not verified:** real-model behaviour. Offline evals prove the guardrails, not reasoning quality; the OpenAI adapter is tested against a local Responses-format server.